

-0 / +9 lines of 169 · 2 parallel region(s) added · 1 of 1 candidate(s) kept · restructuring level 0

BEFORE	AFTER	file
<pre>79 { 80 int i, j, k; 81 82 #pragma scop 83 /* D := alpha*A*B*C + beta*D */</pre>	<pre>79 { 80 int i, j, k; 81 82 #pragma scop 83 /* D := alpha*A*B*C + beta*D */ 84 /* First nest: compute tmp = A*B. 85 Parallelize (i,j) pairs; each pair computes independent tmp[i][j]. 86 k-loop is sequential accumulation within each pair. */ 87 #pragma omp parallel for collapse(2) private(k) firstprivate(alpha, ni, nj, nk 88 for (i = 0; i < _PB_NI; i++) 89 for (j = 0; j < _PB_NJ; j++) 90 { 91 tmp[i][j] = 0; 92 for (k = 0; k < _PB_NK; ++k) 93 tmp[i][j] += alpha * A[i][k] * B[k][j]; 94 } 95 96 /* Second nest: compute D = beta*D + tmp*C. 97 Parallelize (i,j) pairs; each pair computes independent D[i][j]. 98 k-loop is sequential accumulation within each pair. */ 99 #pragma omp parallel for collapse(2) private(k) firstprivate(beta, ni, nj, nl) 100 for (i = 0; i < _PB_NI; i++) 101 for (j = 0; j < _PB_NL; j++) 102 { 103 D[i][j] *= beta; 104 for (k = 0; k < _PB_NJ; ++k)</pre>	
<pre>84 for (i = 0; i < _PB_NI; i++) 85 for (j = 0; j < _PB_NJ; j++) 86 { 87 tmp[i][j] = 0; 88 for (k = 0; k < _PB_NK; ++k) 89 tmp[i][j] += alpha * A[i][k] * B[k][j]; 90 }</pre>		
<pre>91 for (i = 0; i < _PB_NI; i++) 92 for (j = 0; j < _PB_NL; j++) 93 { 94 D[i][j] *= beta; 95 for (k = 0; k < _PB_NJ; ++k)</pre>		